Contenido
Introducción
Características
Bloques de código
Comentarios
Convenciones de formato
Tipos de datos
Casting
Constantes
Control de flujo
Bucles
Funciones
Ámbito de variables
Variables variables
Control de errores
Redirecciones
Inclusiones

Introducción al lenguaje PHP

Este documento cubre los conceptos fundamentales de PHP para el desarrollo web en entorno servidor. Desde las características del lenguaje hasta el manejo de errores y datos.

Características principales

🌱 Open source

✍️ Sencillez

🌐 Multiplataforma

🧩 Orientado a objetos (v5)

🚨 Gestión de errores (excepciones)

📄 Lectura de XML

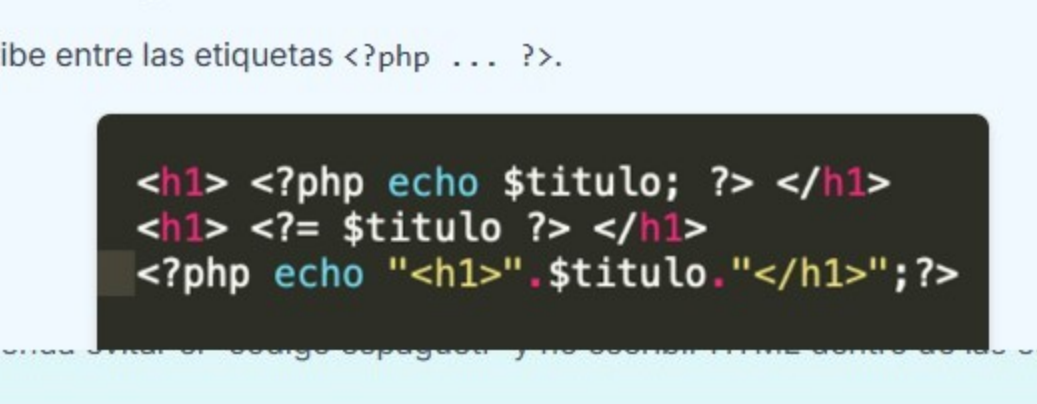
🗄️ Acceso a Bases de datos

🔌 Conexión con otros servicios

🛒 Sistemas de comercio electrónico

Bloques de código PHP

El código PHP se escribe entre las etiquetas `<?php ... ?>`.



```
<h1> <?php echo $titulo; ?> </h1>
<h1> <?= $titulo ?> </h1>
<?php echo "<h1>".$titulo."</h1>";?>
```

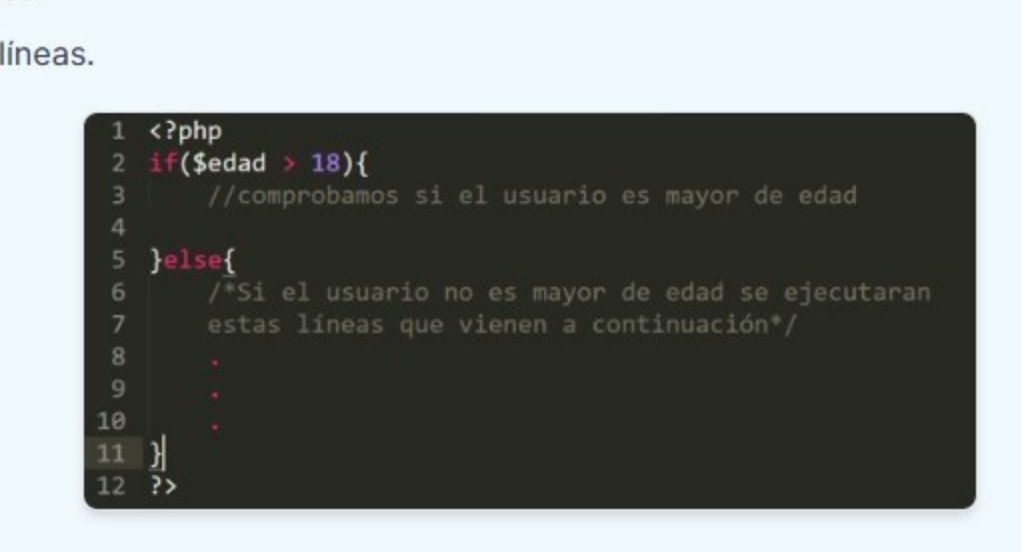
Se recomienda evitar el "código espagueti" y no escribir HTML dentro de las etiquetas PHP.

Comentarios

Para hacer comentarios en PHP, se usa:

`//` o `#` para una sola línea.

`/* ... */` para varias líneas.



```
<?php
if($edad > 18){
    //comprobamos si el usuario es mayor de edad

}else{
    /*Si el usuario no es mayor de edad se ejecutaran
    estas líneas que vienen a continuación*/
    .
    .
    .
}
?>
```

Convenciones de formato

Distinguimos entre varios tipos de escritura para nombrar elementos:

- 🐫 PascalCase/UpperCamelCase : `MiElemento`
- 🐪 camelCase/lowerCamelCase : `miElemento`
- 🐍 snake_case : `mi_elemento`
- 🍢 kebab-case : `mi-elemento`

Reglas comunes:

- Funciones y variables: snake_case
- Métodos: camelCase
- Clases: PascalCase

Se recomienda usar una indentación consistente, ya sean espacios* o tabulaciones. Ver video

Tipos de datos y variables

PHP es un lenguaje historicamente no tipado o de tipado dinámibo,lo que significa que no es necesario especificar el tipo de dato de una variable, y se determinará en tiempo de ejecucióon, esto, además permite cambiar el tipo de una variable dinámicamente.

Desde la versión 7.01 ha habido algunas modificaciones que permiten declarar tipos en según qué datos

Más info sobre los tipos de datos

Eso no significa que no existan los tipos de datos:
- Booleanos
- Números enteros (Integers)
- Números de punto flotante (Floats)
- Cadenas de caracteres (Strings)
- Objetos
- NULO

Casting

Puedes forzar la conversión de tipos usando el casting.

```
<?php
    $foo = 10; // integer
    $bar = (boolean) $foo; // boolean
    $str = "$foo"; // string
    $fst = (string) $foo; // string
?>
```

Constantes

Se declaran en mayúsculas y no pueden ser modificadas.

```
define("CONSTANTE", "Hola mundo.");
const CONSTANTE = 'Hola Mundo';
```

Control de flujo: Condicionales

Se usan las estructuras `if`, `if...else`, `if...elseif...else` y `switch...case` para ejecutar código bajo ciertas condiciones.

```
// IF

if (condicion) {
    // instrucciones
}

// IF ... ELSE

if (condicion) {
    // instrucciones
} else {
    // instrucciones
}

// IF ... ELSEIF ... ELSE

if (condicion)
{
    // instrucciones
}elseif(condición2){
    // instrucciones
}else{
     // instrucciones
}

// SWITCH CASE

switch (variable) {
    case 0:
        // instrucciones
        break;
    case 1:
        // instrucciones
        break;
    default:
        // instrucciones
}
```

Bucles

Para repetir bloques de código, se usan los bucles:

- `for` : Ideal cuando conoces el número de iteraciones.
- `foreach` : Para recorrer arrays.
- `while` : Se ejecuta mientras la condición sea verdadera.
- `do...while` : Se ejecuta al menos una vez, y luego evalúa la condición.

```
// FOR
for ($i = 0; $i < count($lista); $i++) {
    echo $lista[$i];
}

// FOREACH
foreach ($lista as $elemento) {
    echo $elemento;
}

// WHILE
$numero = 1;
while ($numero < 10) {
    echo $numero;
    $numero++;
}

// DO...WHILE
$numero = 1;
do {
    echo $numero;
    $numero++;
} while ($numero < 10);
```

Funciones

Una función es un bloque de código reutilizable. Pueden tener parámetros y devolver valores.

Función simple

Sin parámetros ni retorno.

```
// Función sin parámetros
function saluda() {
    echo "Hola, caracola";
}
saluda();
```

Por valor y por retorno

Con parámetros por valor y un valor de retorno.

```
// Función con parámetro por
valor
function saluda($nombre) {
    echo "Hola, " . $nombre;
}
saluda('Pepe');

// Función con return
function sumaNumeros($a, $b)
{
    return ($a + $b);
}
$resultado = sumaNumeros(3,
10);
```

Por referencia

El parámetro modifica la variable original.

```
// Función con parámetros po
r referencia
function sumaNumeros(&$n1, &
$n2, &$resultado) {
    $resultado = $n1 + $n2;
}

$resultado = 0;
sumaNumeros(3, 30, $resultad
o);
// $resultado ahora es 33
```

Por defecto

Los parámetros pueden tener valores predefinidos.

```
// Función con parámetros por defecto
function datosAlumno($nombre, $telefono, $nota = 0) {
    // ...
}

// Posibles llamadas
datosAlumno('Marián', '6584', 8);
datosAlumno('Marián', '6584');
```

Funciones Nativas

PHP incluye una gran cantidad de funciones predefinidas para manejar textos, fechas, ficheros, etc.

Textos	Fecha y hora	Expresiones regulares	Manejo de ficheros
Ver documentación	Ver documentación	Ver documentación	Ver documentación

`strlen`, `substr`
`strtoupper`, `strtolower`
`trim`, `ltrim`, `rtrim`
`strcmp`, `str_replace`
	`time`, `checkdate`
	`date`
		`ereg`, `ereg_replace`
		`split`
			`Readfile`, `file`
			`file_get_contents`
			`file_put_contents`
			`file_exists`, `filesize`

Ámbito de las variables

El ámbito de una variable determina dónde puede ser accedida. PHP define tres ámbitos principales para las variables:

Variables Locales

Las variables declaradas dentro de una función tienen un ámbito local y no se pueden acceder fuera de esa función.

```
<?php
function test() {
    $a = 1; // Ámbito local
    echo $a;
}
test(); // Muestra 1
// echo $a; // Esto daría un
error
?>
```

Variables Globales

Las variables globales se pueden acceder desde cualquier ámbito usando la palabra clave `global` o el array `$GLOBALS`.

```
<?php
$a = 1; // Ámbito global
function suma() {
    global $a;
    $b = 2; // Ámbito local
    $b = $a + $b; // Accede
a la variable global $a
}
suma();
echo $b; // Muestra 2
// O usando $GLOBALS
$GLOBALS['b'] = $GLOBALS
['a'] + $GLOBALS['b'];
?>
```

Variables Estáticas

Una variable estática conserva su valor entre llamadas a una función, en lugar de ser destruida al finalizar.

```
<?php
function contador() {
    static $a = 0;
    echo $a;
    $a++;
}
contador(); // Muestra 0
contador(); // Muestra 1
contador(); // Muestra 2
?>
```

Variables

Una variable variable es una variable cuyo nombre se puede determinar y cambiar dinámicamente. Esto es útil para acceder a variables cuyo nombre se construye en tiempo de ejecución.

```
<?php
    $texto_es = "Bienvenido";
    $texto_en = "Welcome";
    $idioma = "es";
    $texto = "texto_" . $idioma; // $texto ahora contiene el string "texto_es"
    echo $$texto;
?>
```

En este ejemplo, `$$texto` se evalúa en dos pasos:
1. PHP toma la variable `$texto` y obtiene su valor, que es la cadena `"texto_es"`.
2. Luego, PHP busca una variable cuyo nombre sea `"texto_es"`, que en este caso es `$texto_es`.
El resultado final es el valor de `$texto_es`, que es "Bienvenido". Es una forma avanzada de programación que te permite usar el valor de una variable para obtener el nombre de otra.

Control de errores

Se pueden manejar los errores para mejorar la experiencia del usuario y evitar que se muestren mensajes de error feos.

- `@` : Operador que se coloca delante de una expresión para suprimir los errores que pueda generar.
- `die()` : Detiene la ejecución del script.
- Excepciones : Usando los bloques `try...catch`, que son una forma estructurada y moderna de manejar errores.

```
try {
    if (!file_exists("fich1.txt")) {
        throw new Exception("El fichero de entrada no existe");
    }
    $contenido = file_get_contents("fich1.txt");
} catch (Exception $e) {
    echo 'Se ha producido un error: ' . $e->getMessage();
}
```

Redirecciones

Se utilizan para enviar al usuario a otra página. Deben ejecutarse **siempre** antes de que se envíe cualquier contenido al navegador.

```
<?php
header("Location: login.php");
// o con un retardo
header("Refresh: 5; url=index.php");
exit()
?>
```

El uso de `exit()` después de una redirección es una buena práctica para asegurarse de que no se ejecute ningún código adicional después de la redirección.

En el futuro hablaremos más sobre las redirecciones.

Inclusiones de Archivos

Para incluir y reutilizar código de otros archivos, se usan las siguientes directivas:

contenido al navegador.

```
<?php
header("Location: login.php");
// o con un retardo
header("Refresh: 5; url=index.php");
exit()
?>
```

El uso de `exit()` después de una redirección es una buena práctica para asegurarse de que no se ejecute ningún código adicional después de la redirección.

En el futuro hablaremos más sobre las redirecciones.

Inclusiones de Archivos

Para incluir y reutilizar código de otros archivos, se usan las siguientes directivas:

- `include` : Si el archivo no se encuentra, genera una advertencia (`warning`) y el script continúa.
- `include_once` : Como `include`, pero evita que el mismo archivo se incluya más de una vez.
- `require` : Si el archivo no se encuentra, genera un error fatal (`fatal error`) y detiene la ejecución del script.
- `require_once` : Como `require`, pero evita inclusiones duplicadas.

Este documento se ha realizado por Marián Capilla Herrero. Publicada bajo licencia
Creative Commons Atribución/Reconocimiento-CompartirIgual 4.0 Internacional.
Las modificaciones son permitidas siempre que sean mencionadas, la licencia no se pude modificar

Logo DAW Logo Unión Europea Logo Generalitat Valenciana Logo IES Camp de Morvedre

contenido al navegador.

```
<?php
header("Location: login.php");
// o con un retardo
header("Refresh: 5; url=index.php");
exit()
?>
```

El uso de `exit()` después de una redirección es una buena práctica para asegurarse de que no se ejecute ningún código adicional después de la redirección.

En el futuro hablaremos más sobre las redirecciones.

Inclusiones de Archivos

Para incluir y reutilizar código de otros archivos, se usan las siguientes directivas:

- `include` : Si el archivo no se encuentra, genera una advertencia (`warning`) y el script continúa.
- `include_once` : Como `include`, pero evita que el mismo archivo se incluya más de una vez.
- `require` : Si el archivo no se encuentra, genera un error fatal (`fatal error`) y detiene la ejecución del script.
- `require_once` : Como `require`, pero evita inclusiones duplicadas.

Este documento se ha realizado por Marián Capilla Herrero. Publicada bajo licencia
Creative Commons Atribución/Reconocimiento-CompartirIgual 4.0 Internacional.
Las modificaciones son permitidas siempre que sean mencionadas, la licencia no se puede modificar

Logo DAW Logo Unión Europea Logo Generalitat Valenciana Logo IES Camp de Morvedre